睿抗机器人开发者大赛

CAIP 信息技术创新赛道

AI 视觉应用赛设计文档

作品编号：_____

作品名称：轻量化生活垃圾识别科普分类系统

填写日期：2025 年 10 月 12 日

目 录

第 1 章 可行性分析 1
1.1 技术可行性 1
1.1.1 采用成熟的技术栈 1
1.1.2 模型部署简单 1
1.1.3 代码结构清晰 1
1.2 经济可行性 1
1.3 社会效益可行性 2
1.4 运行环境可行性 2
1.5 政策和资源可靠性 2

第 2 章 产品架构设计 3
2.1 作品技术路线图 3
2.2 总体设计架构图 4
2.3 模块划分 4
2.3.1 图像采集模块 4
2.3.2 模型推理模块 5
2.3.3 结果处理模块 5
2.3.4 数据管理模块 5
2.3.5 训练优化模块 5
2.3.6 系统管理模块 5
2.3.7 模块的类图/时序图 6
2.3.8 接口设计 6

第 3 章 核心算法与优化训练 9
3.1 YOLO 系列算法 9
3.2 使用 ultralytics 库进行 YOLO 模型训练 11
3.3 基于 PyTorch 的 GPU 加速 (torch.cuda) 12
3.4 MLflow 实验跟踪和管理 12
3.5 自定义损失函数 (CustomLossCalculator) 和类别权重 12
3.6 Albumentations 库进行高级数据增强 13
3.7 多进程数据加载 (workers=0) 14
3.8 Adam 优化器 14
3.9 性能优化 15

第 4 章 模块展示 16
4.1 图像采集模块 16
4.2 模型推理模块 17
4.3 结果处理模块 17
4.4 数据管理模块 19
4.5 训练优化模块 20
4.6 系统管理模块 21

第 5 章 落地案例 23

5.1 源启星链（广西）科技有限公司 24
5.2 上海势曦苑科技有限公司 25
5.3 温州富兴阀门有限公司 26
5.4 鹿寨乾元德信科技服务中心 27
5.5 广西跨境工厂科技有限公司 28
5.6 北海市环境卫生管理处 29
第 6 章 作品总结 31
第 7 章 作品展望 32
参考文献 34

第 1 章 可行性分析

1.1 技术可行性

1.1.1 采用成熟的技术栈

表格 1 技术选用介绍表

技术名称	选型依据
YOLO 模型	使用 YOLO（You Only Look Once）系列目标检测模型，本模型在工业界和学术界广泛应用，具备高精度与实时性[1]
PyQt5 图形界面框架	PyQt5 是 Python 中广泛使用的 GUI 开发工具包，适合开发桌面应用程序，学习曲线平缓，社区资源丰富
OpenCV 图像处理库	用于图像采集、预处理、绘制检测框等操作，是计算机视觉领域的标准库之一[2]

1.1.2 模型部署简单

模型训练可在本地或云平台完成（如 Google Colab），训练完成后导出 .pt 文件即可直接部署到本地程序中。模型推理部分只需加载权重文件，不需要高性能服务器，普通笔记本电脑即可运行。

1.1.3 代码结构清晰

整个项目模块化设计良好，包括摄像头线程、图像处理、UI 控件管理等，便于维护和扩展；同时使用多线程机制来处理视频流和模型推理，避免界面卡顿，提升了用户体验。

1.2 经济可行性

传统智能分类设备单台成本超 5 万元，而基于 YOLOv11 的方案仅需低成本边缘设备（例如 AI+教学实验平台）即可实现同等精度.以改进 YOLOv5s 为例，参数量压缩后模型体积缩小至原 77.9%，显著降低硬件算力需求。此外，开源框架（如 PyTorch、TensorFlow）和社区资源的丰富性可减少开发成本，且通过迁

移学习技术，可利用现有垃圾数据集（如 URPC、RUOD）进行微调，避免从零构建数据集的高昂投入。长期运维中，智能识别可减少人工督导成本（占传统模式运营成本的 60%）。

1.3 社会效益可行性

通过实时检测与错误投放记录的反馈机制，可推动分类行为从被动监管向主动参与转变，助力环保体系建设。据北京市“十四五”规划，生活垃圾无害化处理率需稳定在 100%，本项目通过提升分类准确率间接促进资源回收率提升，减少填埋与焚烧带来的碳排放。此外，项目可作为教育载体，通过数据可视化（如 Grad-CAM）增强学习者环保认知。

1.4 运行环境可行性

基于垃圾分类场所的特殊性质，光照变化、遮挡干扰等复杂因素会大幅度影响检测环境。可通过数据增强技术（如随机光照调整、遮挡模拟）缓解。YOLOv11 的 CARAFE 上采样模块和动态非单调聚焦系数优化技术提升了对多角度目标的适应性。硬件层面，设备算力约束（如 CPU≤2.0GHz、内存≤16GB）与 YOLOv11 的轻量化设计匹配，且相机分辨率上限（600 万像素）与常见摄像头规格一致。数据采集端，基于赛事组委会通知，引入项目团队采集数据与项目实践单位数据集，符合赛事规则。

1.5 政策和资源可靠性

《关于加快推进部分重点城市生活垃圾分类工作的通知》（住建部，2017）[3]与《高等学校人工智能创新行动计划》（教育部，2018）[4]明确支持垃圾分类技术研发与教育应用。

2023 年《算力基础设施高质量发展行动计划》提出公共算力资源覆盖校园，为项目提供算力平台支撑。此外，赛事推荐的边缘计算设备（如 AI+教学实验平台）与 YOLOv11 的部署需求高度契合，确保技术方案的可实施性[5]。

同时，本团队联系当地市环卫局，根据当地政府部分政策文件支持方向进行项目研究。

第 2 章 产品架构设计

2.1 作品技术路线图

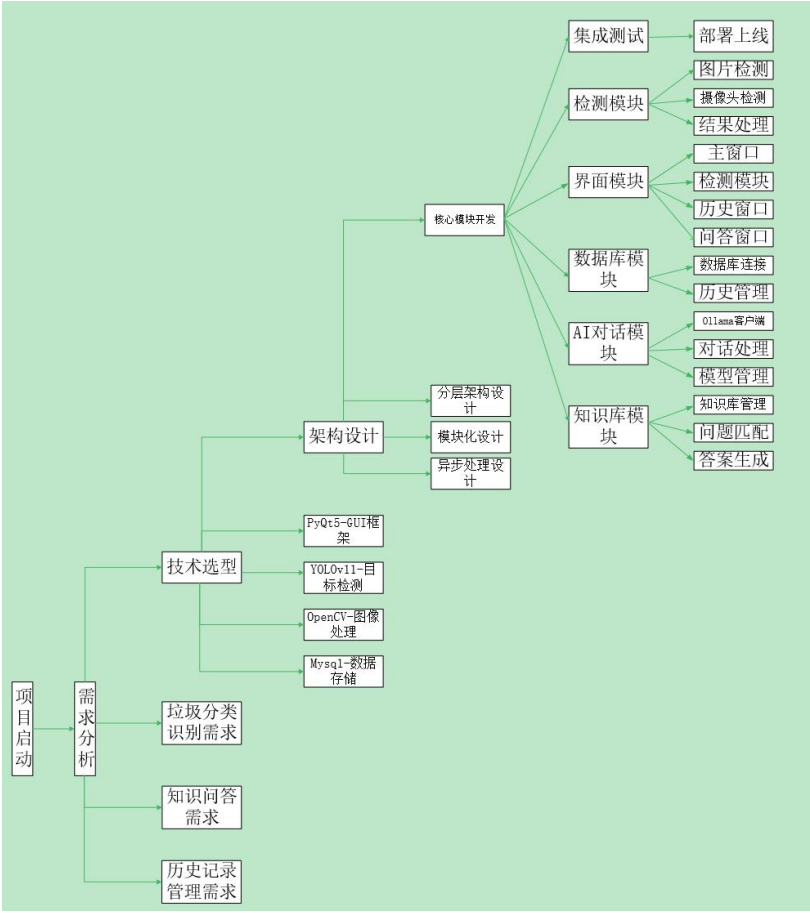


图 1 作品技术路线图

2.2 总体设计架构图

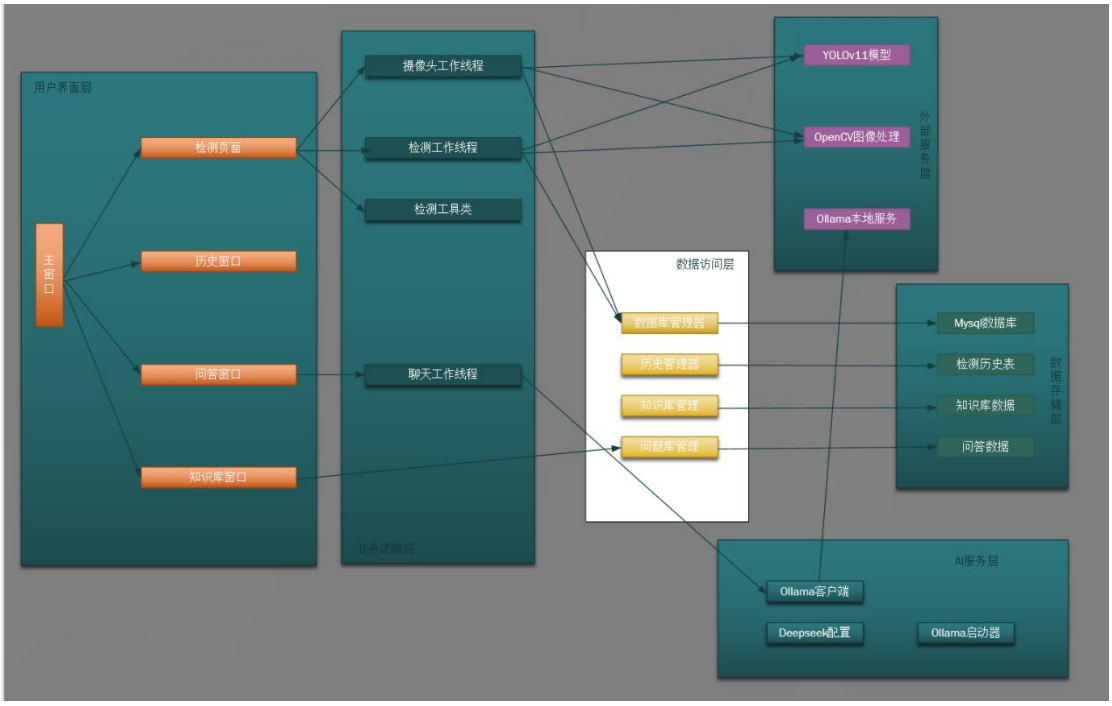


图 2 总体设计架构图

2.3 模块划分

2.3.1 图像采集模块

采用**多源异构数据采集架构**，通过静态图像加载、实时视频流采集与智能帧缓冲机制的协同设计，构建面向各类现实垃圾分类场景的动态数据供给体系。

静态图像加载：系统提供标准化文件对话框接口，支持用户从本地存储设备选择 PNG/JPG 格式的静态样本。

界面集成双窗格布局：可实时显示选中图像的 EXIF 元数据（分辨率、拍摄时间等）。该功能主要用于离线测试与历史数据复用，特别针对快递纸箱压痕等特殊样本的标注训练。

实时视频流采集：基于 WebcamWorker 类实现的多线程采集框架，采用生产者-消费者模型分离视频捕获与预处理流程。底层通过 OpenCV 的 V4L2 驱动接口直接访问 USB 摄像头硬件，以 60FPS 速率获取 640×480 分辨率的 RGB 帧数据。

智能帧缓冲机制：采用 deque 双端队列构建三级缓存池（最大容量 3 帧），

通过滑动窗口策略实现视频流的时空对齐优化，提高稳定性。

2.3.2 模型推理模块

使用 YOLOv11n 作为基础检测模型，自定义预处理流程，通过“保持宽高比缩放+边缘填充”保证检测物检测录入尽可能完整。

使用多帧稳定性检测机制通过历史检测结果过滤抖动；使用 GPU 加速利用 CUDA 流处理提升性能。

2.3.3 结果处理模块

首先使用 IOU 计算改进边界框匹配算法；然后通过稳定性过滤多帧选择一致性判断检测物品种类；经过可视化渲染绘制检测框及标签信息。

最后自动/手动保存（可支持多种格式保存检测结果）。

2.3.4 数据管理模块

通过参数配置变动调节置信度阈值，更改检测物检测阈值，使“检测精度与置信度阈值”正相关变动；检测数量与置信度阈值”负相关变动。

设置持久化将主题、自动保存等设置保存；随后将结构化数据导出（JSON/TXT）生成报告（在垃圾分类知识库使用中英文映射表进行提示）。

2.3.5 训练优化模块

使用自定义损失函数引入类别权重平衡数据分布；应用多种 Albumentations 变换增强数据策略；使用 MLflow 进行实验跟踪；最后模型评估记录 mAP 等关键指标。

2.3.6 系统管理模块

建立启动/停止检测任务的入口、处理用户交互、管理工作线程的生命周期以及更新 UI 状态的逻辑。

将耗时的检测任务从主 UI 线程分离出来执行。它们负责具体任务（采集帧或加载图片、进行检测）的执行、管理自身线程的生命周期、资源（如摄像头）的获取和释放，并通过信号报告进度、结果和错误。

确保后台线程的处理状态能够正确反映到用户界面上，用户操作也能正确触发后台任务。

根据任务状态（是否正在检测），启用或禁用相关的操作按钮（如“开始检测”、“保存结果”等）。

2.3.7 模块的类图/时序图

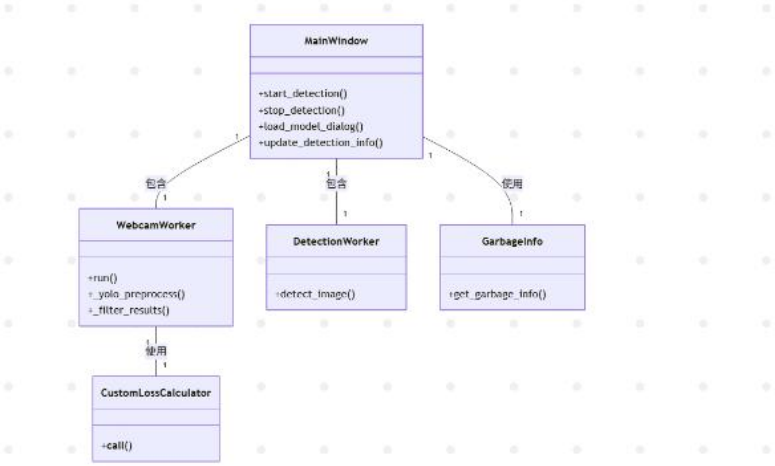


图 3 类图

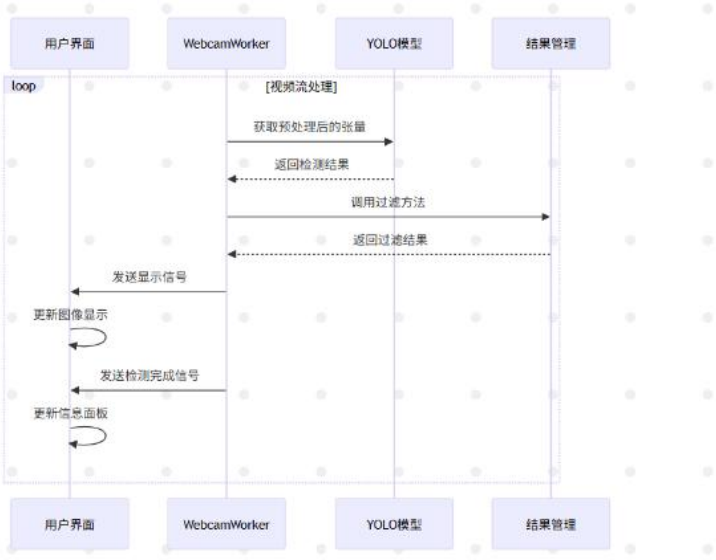


图 4 主要时序图

2.3.8 接口设计

可视化界面接口设计：

（1） 使用 ultralytics.YOLO 模型推理接口在 GUI 应用中加载训练好的模型，并对新的图片或视频帧进行目标检测。

YOLO(model_path)初始化 YOLO 模型，加载权重，是加载模型的接口。

model.predict(source, conf=...)进行实际推理检测的接口，输入图像/帧数据(source)，输出检测结果对象。通过 conf 参数调整置信度阈值。

（2） PyQt5 GUI 控件接口 (信号与槽)实现用户操作（如点击按钮、滑动滑块、选择下拉框）与后台逻辑之间的交互，以及后台处理结果更新 UI 的机制。

按钮的 clicked 信号连接到对应的槽函数（如 model_button.clicked.connect(self.load_model_dialog)）。

下拉框的 currentIndexChanged 信号连接到槽函数（如 detect_type_combo.currentIndexChanged.connect(self.on_detect_type_changed)）.

滑块的 valueChanged 信号连接到槽函数（如 conf_slider.valueChanged.connect(self.update_conf_threshold)）。

自定义工作线程 (QThread) 使用 pyqtSignal 定义信号（如 WebcamWorker.frame_ready, DetectionWorker.detection_complete），在耗时操作完成后发出，连接到主线程的槽函数更新 UI。

（3） QThread 工作线程接口将耗时的检测任务（图片检测、摄像头实时检测）放到独立的线程中执行，避免阻塞主 UI 线程，保证界面的流畅性。

WebcamWorker(model, conf_threshold) / DetectionWorker(model, detection_type, source, conf_threshold): 线程类的构造函数，接收模型、检测类型、源等参数。

worker.start(): 启动线程的接口。

worker.stop(): 提供一个方法来安全停止线程（通过设置内部标志并等待线程结束）。

frame_ready = pyqtSignal(object): 定义信号，用于线程将处理好的帧数据发送回主线程。

detection_complete = pyqtSignal(object): 定义信号，用于线程发送检测结果对象回主线程。

error_occurred = pyqtSignal(str): 定义信号，用于线程报告发生的错误。

（4） 文件系统交互接口允许用户选择模型文件、输入图片，以及保存检测结

果和报告。

QFileDialog.getOpenFileName(...): 提供标准的“打开文件”对话框接口。

QFileDialog.getSaveFileName(...): 提供标准的“保存文件”对话框接口。

cv2.imwrite(path, image): 使用 OpenCV 接口将图像数据保存到文件。

os.makedirs(path, exist_ok=True): 使用 OS 接口创建保存所需的目录。

文件读写（用于报告导出，如 open(save_path, 'w', ...) 和 json.dump(...)）。

（5） 摄像头硬件接口将访问和控制本地摄像头以获取视频流进行实时检测。

cv2.VideoCapture(0, cv2.CAP_DSHOW): 使用 OpenCV 接口打开默认摄像头（或指定索引）。

cap.isOpened(): 检查摄像头是否成功打开。

cap.set(property, value): 设置摄像头属性（如分辨率、帧率）。

cap.read(): 读取摄像头的下一帧。

cap.release(): 释放摄像头资源。

（6） 内部数据处理和展示接口来处理检测结果、更新 UI 上的图像和文本信息。

display_webcam_result(frame) / handle_detection_results(results): 槽函数，接收工作线程发来的数据，并更新 QLabel 显示图像。

update_detection_info(results): 处理检测结果对象，提取统计信息，并格式化显示在 QTextEdit 中。

get_garbage_info(class_name): 一个内部接口（函数），用于根据英文类别名称查找中文名称、分类和处理建议的映射信息。

manual_save_result(): 用户手动触发的保存当前显示结果的接口。

save_results(): 保存图片检测结果的接口。

show_statistics(): 显示检测统计信息对话框的接口。

export_report(): 导出检测报告文件的接口。

handle_auto_save(frame, result): 处理自动保存请求的内部接口。

训练模型代码接口设计：

（1） ultralytics.YOLO 模型训练接口:自动执行端到端的模型训练流程。

YOLO(...).to(device): 初始化模型并将其放到指定设备。

model.train(...): 最核心的训练接口，接收大量参数配置训练过程。

model.save(path), model.export(format): 保存和导出训练好的模型的接口。

（2） 自定义损失函数接口 (CustomLossCalculator.__call__)允许在训练循环中插入自定义的损失计算逻辑。

类需要实现 __call__ 方法，接收模型的预测结果和真实标签作为输入，返回计算得到的总损失以及各分量损失。

（3） MLflow 跟踪接口记录训练过程中的参数、指标和文件。

使用 mlflow 库提供的各种 log_params, log_metric, log_artifact 等函数接口。

（4） 数据增强配置接口解决配置训练时应用的数据增强策略。

在 model.train() 方法中通过参数（如 augment, mosaic, mixup, degrees, hsv_h 等）进行配置。虽然代码中定义了一个 create_custom_augmentations 函数，它提供了一个基于 Albumentations 定义增强管道的接口，但实际训练中使用了 Ultralytics 内置的增强配置接口。

第 3 章 核心算法与优化训练

3.1 YOLO 系列算法

YOLO(You Only Look Once) 系列算法:

YOLO 将目标检测任务视为一个回归问题，能够在单次网络前向传播中预测图像中的所有目标及其边界框和类别，实现端到端的高效检测。代码中使用的 ultralytics 库支持最新的 YOLO 版本（如 YOLOv8, YOLOv11 等），具体的版本取决于加载的.pt 模型文件。

实现方式:代码通过 from ultralytics import YOLO 导入，并加载一个.pt 模型文件 (yolo11n.pt)训练过程 (model.train(...)) 调用了 YOLO 模型内部定义的前向传播、损失计算和反向传播逻辑。

自定义损失函数 (CustomLossCalculator) / 加权损失 / 类似 Focal Loss 的调制：

解决的具体问题:衡量模型预测结果与真实标签之间的差异，并指导模型优化。特别是，代码中的自定义损失计算器解决了类别不均衡的问题，通过为样本较少的类别分配更高的权重 (alpha)，并对分类损失应用类似 Focal Loss 的调制 ((1 - torch.exp(-losses[1])) ** self.gamma)，降低模型在易分样本上的损失贡献，使模型更关注难分样本和少数类别，从而提高整体检测性能，尤其是有助于提升少数类别的召回率和精度。

实现方式:定义了一个 CustomLossCalculator 类，它结合了 YOLO 模型原有的边界框损失 (box_loss)、分类损失 (cls_loss) 和 DFL 损失 (dfl_loss)，并根据预设的类别样本分布计算反比例权重应用于分类损失，再通过指数项和 gamma 参数进行调制，最后按自定义权重 (box_weight,cls_weight,dfl_weight) 加权求和得到总损失。这个自定义计算器替换了模型默认的损失计算器。

Adam 优化算法:

解决的具体问题:根据损失函数的梯度信息，迭代地更新模型的可学习参数（权重和偏置），以最小化损失函数，从而提高模型的性能。Adam 是一种自适应学习率优化器，结合了 Momentum 和 RMSprop 的优点，通常收敛速度快且性能稳定。

实现方式:在 model.train()方法中，通过参数 optimizer="Adam"指定使用 Adam 优化器，并配置其学习率 (lr0,lrf) 和权重衰减=(weight_decay) 等参数。

数据增强 (Data Augmentation) 算法:

解决的具体问题:通过对原始训练图像进行各种随机变换（如旋转、翻转、缩放、平移、剪切、改变亮度/对比度、添加噪声、模拟雨/雾等），人工增加训练数据集的多样性和数量。这有助于提高模型的泛化能力和鲁棒性，使其能够更好地适应真实世界中目标可能出现的各种姿态、光照和环境条件。

实现方式:代码在 model.train()中通过设置 augment=True 并配置了多种参数（如 mosaic,mixup,degrees,translate,scale,shear,perspective,flipud,fliplr,hsv_h,hsv_s,hsv_v），启用了 Ultralytics 内置的强大数据增强管道，包括 Mosaic 和 Mixup 等高级增强技术。虽然代码中定义了基于 Albumentations 的自定义增强函数，但在实际的 train 调用中，使用的是 Ultralytics 内置的增强方法。

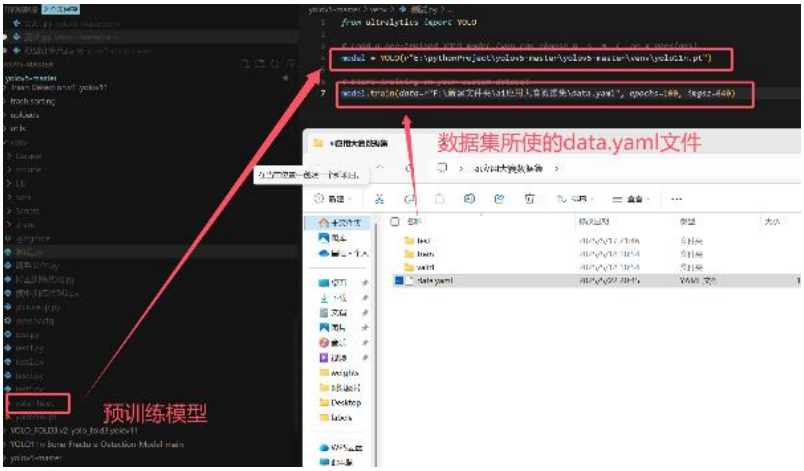


图 5 基础训练模型代码

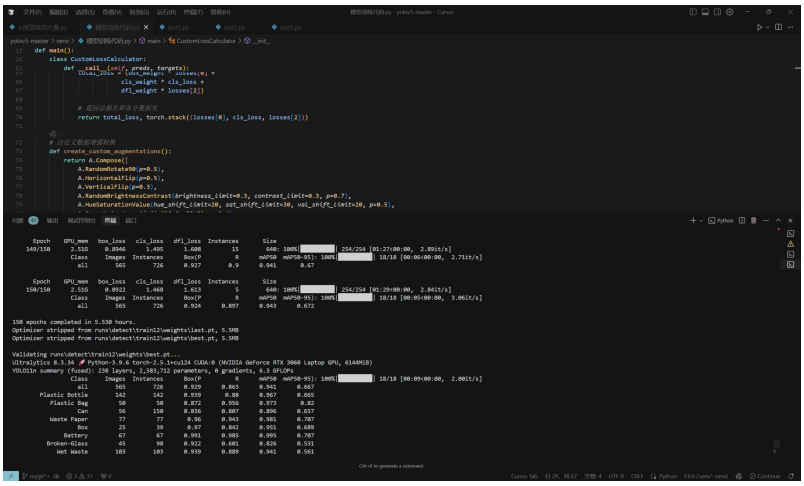


图 6 训练模型（训练结束后结算页面）

3.2 使用 ultralytics 库进行 YOLO 模型训练

解决的具体问题:简化 YOLO 模型的加载、配置和训练流程。ultralytics 库提供了高级 API，抽象了底层 PyTorch 操作、数据加载、分布式训练、模型评估等复杂细节。

工程化实现:

通过 from ultralytics import YOLO 导入库。

使用 model = YOLO(...)加载预训练模型。

直接用 model.train(...)方法启动训练，并通过参数配置训练过程（数据集路径、epochs、batch size、优化器、设备等）。

利用库内置的训练回调（如 after_train）或直接访问 model.trainer 来获取训

练过程中的指标和模型状态。

3.3 基于 PyTorch 的 GPU 加速 (torch.cuda)

解决的具体问题:利用 GPU 的并行计算能力加速模型的推理和训练过程，显著减少计算时间，尤其对于大型模型和数据集。

工程化实现:

使用 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu') 检测并选择可用的 GPU 或回退到 CPU。

通过.to(device)方法将模型和数据（尽管在训练 API 内部处理较多，但原理是将张量移动到指定设备）传输到 GPU 上。

代码中明确打印出正在使用的设备(print(f"Using device: {device}"))。

3.4 MLflow 实验跟踪和管理

解决的具体问题: 系统地记录、组织和比较不同训练运行的参数、指标、代码版本和生成的文件（如模型权重、训练结果）。方便实验回溯、复现和性能分析。

工程化实现:

设置跟踪服务器 URI；

创建或设置实验 ；

创建一个运行上下文，在该上下文中记录信息；

使用 mlflow.log_params(params)记录训练参数；

使用 mlflow.log_artifact(local_path, artifact_path)保存重要文件（如最佳模型、最终结果文件）；

利用 after_train 回调在训练结束后统一记录最终指标和文件。

3.5 自定义损失函数 (CustomLossCalculator) 和类别权重

解决的具体问题:针对特定数据集（如类别不均衡）调整损失函数的计算方式，以改善模型对某些类别（特别是样本较少的类别）的检测性能。这里具体应

用了类别不均衡权重 (alpha) 和 Focal Loss 的思想 (gamma)。

工程化实现:

定义了一个 CustomLossCalculator 类，初始化时接收模型对象。

在__call__方法中，首先调用模型的原始损失计算方法 (self.model.compute_loss(preds, targets)) 获取基础损失分量（box, cls, dfl）。

手动计算并应用反比例的类别权重 (alpha) 到分类损失上，并通过 (1 - torch.exp(-losses[1])) ** self.gamma 应用类似 Focal Loss 的调制系数，降低易分样本的权重。

根据自定义的权重 (box_weight,cls_weight,dfl_weight) 加权组合三个损失分量，得到最终的总损失。

将模型对象的 loss 属性替换为这个自定义损失计算器的实例 (model.loss = CustomLossCalculator(model))。

3.6 Albumentations 库进行高级数据增强

解决的具体问题:通过应用多种图像变换（如旋转、翻转、调整亮度对比度、模糊、噪声、雨雾模拟、剪切、透视、Cutout/CoarseDropout 等）来增加训练数据的多样性，提高模型的泛化能力和鲁棒性，使其更能适应不同环境条件下的图像。

工程化实现:

通过 import albumentations as A 导入库。

定义 create_custom_augmentations()函数，使用 A.Compose 组合多种增强变换，并设置概率 p 控制每种变换的应用几率。

配置 A.BboxParams(format='yolo', label_fields=['class_labels'])确保边界框坐标在增强时能正确地与图像同步变换，且使用 YOLO 格式。

虽然代码中定义了自定义增强，但在 model.train 调用时，直接设置了 augment=True 并配置了 Ultralytics 内置的多种增强参数 (mosaic,mixup,degrees,translate,scale,shear,perspective,flipud,fliplr,hsv_h,hsv_s,hsv_v)。这意味着代码实际使用了 Ultralytics 内置的增强管道，而不是上面定义的 Albumentations 管道。这是一个值得注意的点：自定义的 create_custom_augmentations 函数被定义了但未在 model.train 中直接使用，而是

依赖了 Ultralytics 的内置增强。如果希望使用 Albumentations，需要通过 Ultralytics 的配置或自定义数据加载器来集成。

3.7 多进程数据加载 (workers=0)

解决的具体问题:在数据加载（读取图片、增强、预处理）成为训练瓶颈时，使用多个进程并行加载数据，确保 GPU 能够持续获得数据进行训练，减少 GPU 空闲时间。

工程化实现:

在 model.train 方法中设置 workers=0.注意:在 Windows 系统上，将 workers 设置为大于 0 的值可能会遇到兼容性问题（如 RuntimeError: An attempt has been made to start anew process before the current process has finished its bootstrapping phase.），因此通常在 Windows 上设置为 0，表示使用主进程加载数据。在 Linux/macOS 上可以设置为大于 0 的值以利用多进程。代码中通过 torch.multiprocessing.freeze_support()在 if __name__ == '__main__':块内调用 main() 来兼容 Windows 的多进程启动机制，但这只为潜在的多进程使用做准备，实际训练参数 workers=0 关闭了多进程数据加载。

3.8 Adam 优化器

解决的具体问题:使用一种自适应学习率的优化算法来更新模型权重，Adam 结合了 Momentum 和 RMSprop 的优点，通常收敛速度较快且对学习率不那么敏感。

工程化实现:

在 model.train 方法中设置 optimizer="Adam"。

同时配置 Adam 优化器的相关参数，如初始学习率 (lr0)、最终学习率比率 (lrf)、权重衰减 (weight_decay) 以及 Adam 特有的动量参数 beta1 和 beta2(这些参数在 Ultralytics 的训练函数内部被解析和应用)。

3.9 性能优化

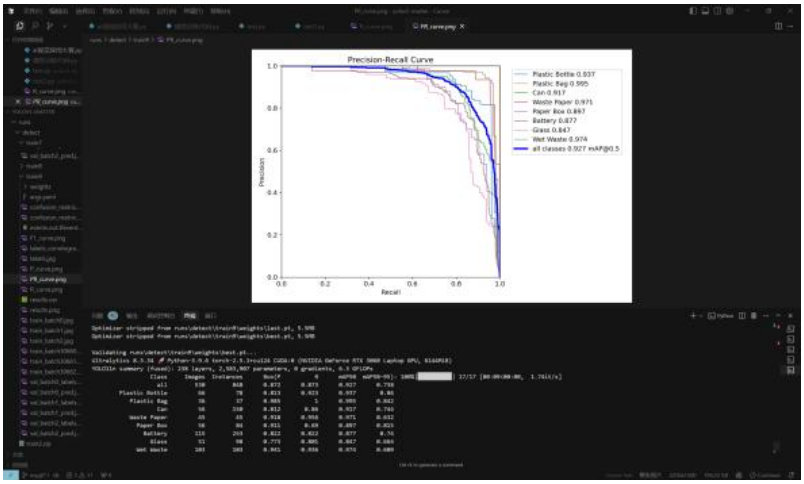


图 7 第一次训练模型结果

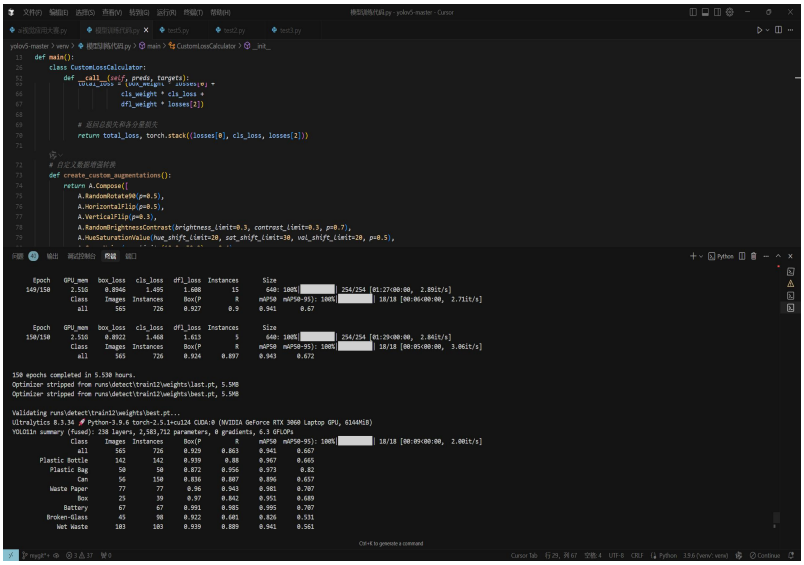


图 8 第二次模型训练结果

从核心的 Precision、Recall 和 mAP@0.5 指标来看，第二次训练（使用了优化技术）的模型在整体检测性能上优于第一次训练的模型。尤其是在查全率方面有显著改进，这对于垃圾识别应用来说是很重要的，意味着能漏检更少的垃圾。尽管 mAP@0.5:0.95 指标有所下降，但考虑到主要的识别任务对 mAP50 更敏感，并且训练 Epoch 数减少了，这次优化仍然可以认为是成功的，因为它在提高主要性能指标的同时减少了训练所需的 Epoch 数。

第 4 章 模块展示

4.1 图像采集模块

静态图像加载：系统提供标准化文件对话框接口，支持用户从本地存储设备选择 PNG/JPG 格式的静态样本（如图 3 所示）。

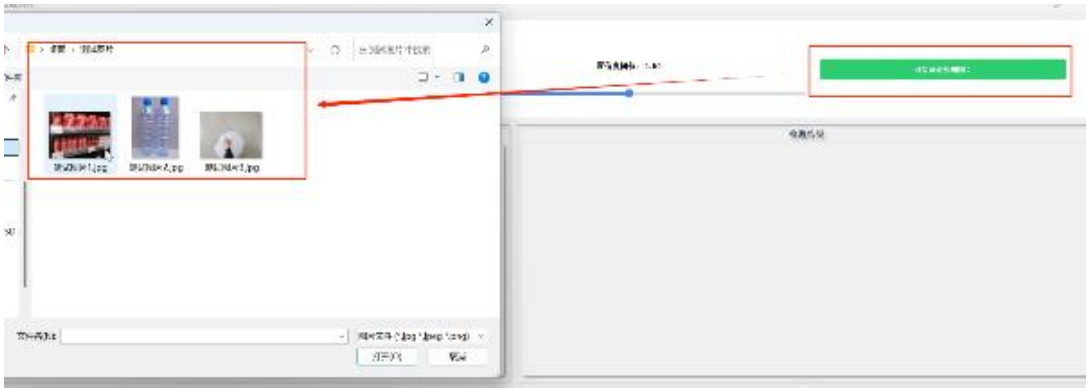


图 9 静态图像采集

实时视频流采集：基于 WebcamWorker 类实现的多线程采集框架（图 4 界面显示双画面同步输出），采用生产者-消费者模型分离视频捕获与预处理流程。底层通过 OpenCV 的 V4L2 驱动接口直接访问 USB 摄像头硬件，以 60FPS 速率获取 640×480 分辨率的 RGB 帧数据。

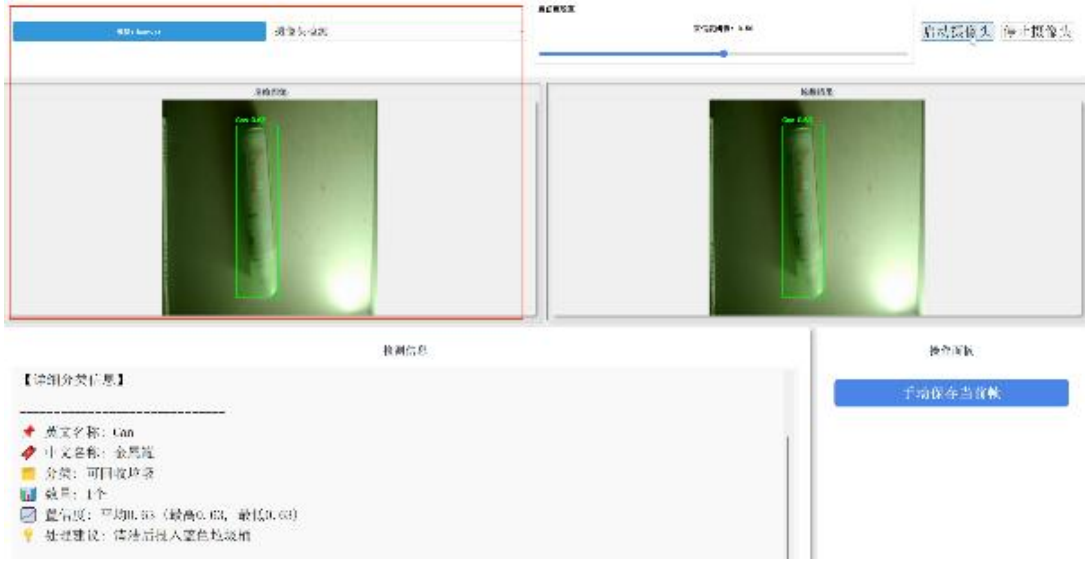


图 10 实时视频流采集

功能说明：通过 `data_path` 指定数据集路径（YOLO 格式的 `data.yaml` 文件）；使用 `albumentations` 库实现复杂的数据增强（如旋转、翻转、亮度调整等），提升模型泛化能力；通过 `bbox_params` 定义边界框标注格式（YOLO 格式）。

4.2 模型推理模块

使用 YOLOv11n 作为基础检测模型，自定义预处理流程，通过“保持宽高比缩放+边缘填充”保证检测物检测录入尽可能完整（可参考图 5）。

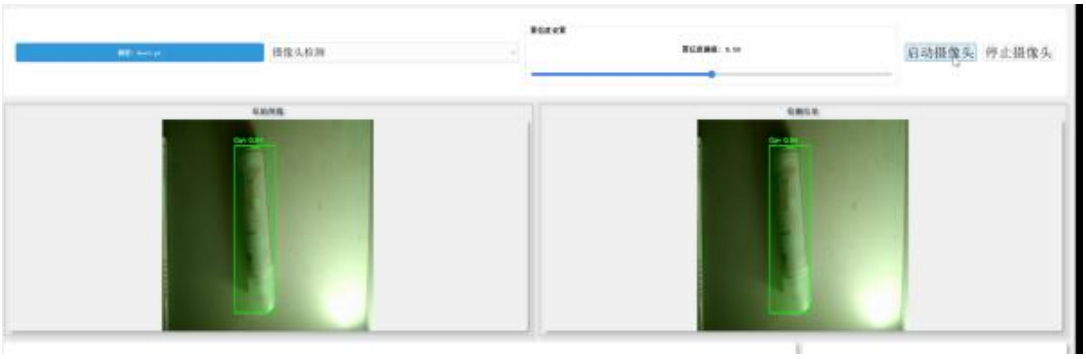


图 11 检测物检测

使用多帧稳定性检测机制通过历史检测结果过滤抖动；使用 GPU 加速利用 CUDA 流处理提升性能。

功能说明：加载 YOLOv11 预训练模型（`yolov11n.pt`）;在训练完成后调用 `after_train` 函数，记录验证集的 mAP 指标（隐含模型推理过程）;保存最佳模型权重（`best_model_path`）。

4.3 结果处理模块

首先使用 IOU 计算改进边界框匹配算法；然后通过稳定性过滤多帧选择一致性判断检测物品种类；经过可视化渲染绘制检测框及标签信息。

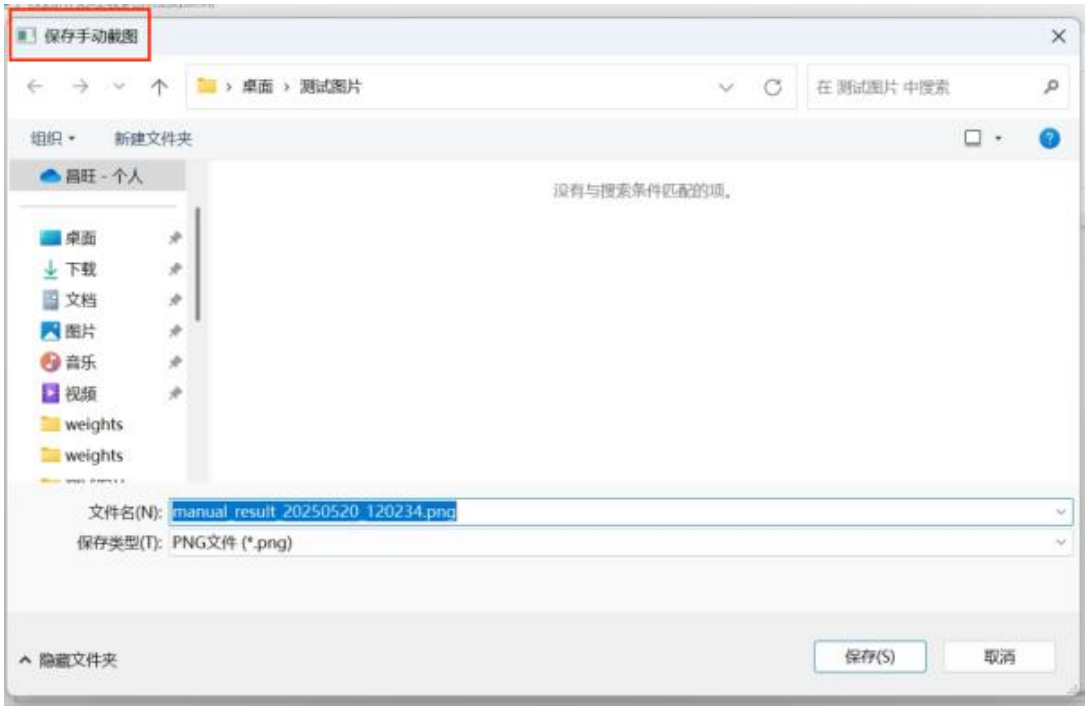


图 12 手动保存

最后自动/手动保存（可支持多种格式保存检测结果）。

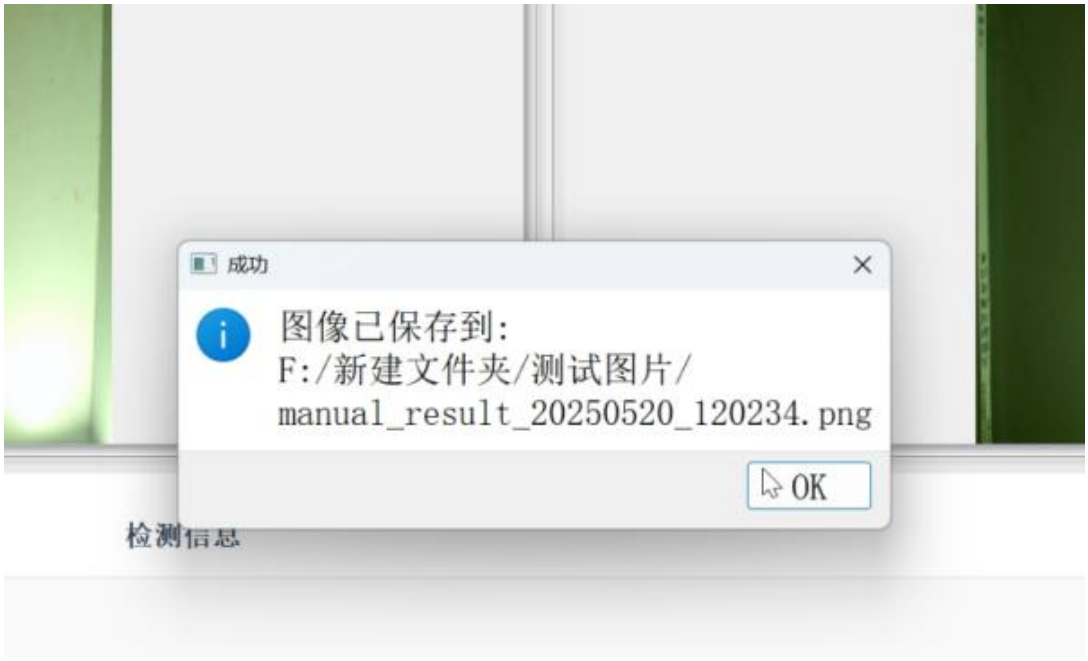


图 13 结果保存

功能说明：使用 `mlflow` 记录验证集 mAP50、mAP5095 等指标；按类别记录每个类别的 AP 值（如 `AP_Plastic Bottle`）；导出模型为 TorchScript 格式，并通过 MLflow 保存为 artifact。

4.4 数据管理模块

可通过参数配置变动调节置信度阈值，更改检测物检测阈值，使“检测精度与置信度阈值”正相关变动；检测数量与置信度阈值”负相关变动。

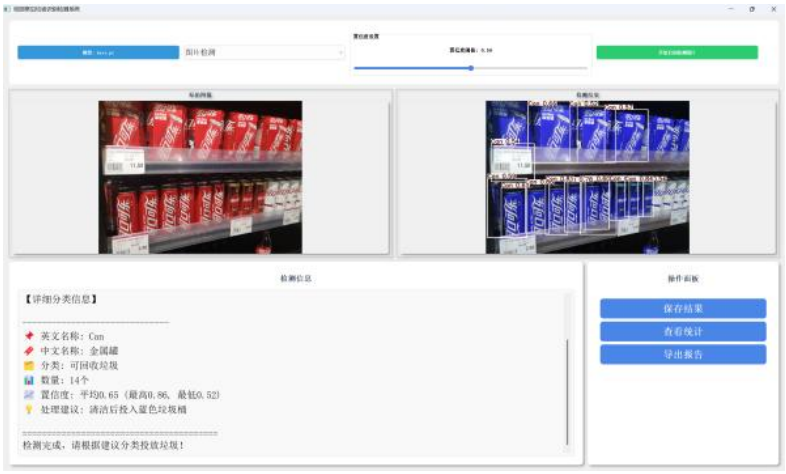


图 14 检测物置信度50%

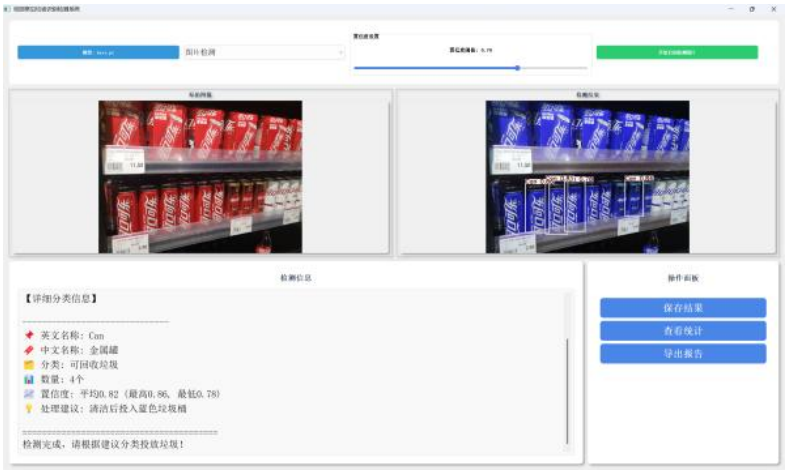


图 15 检测物置信度70%

设置持久化将主题、自动保存等设置保存；随后将结构化数据导出（JSON/TXT）生成报告（在垃圾分类知识库使用中英文映射表进行提示）。

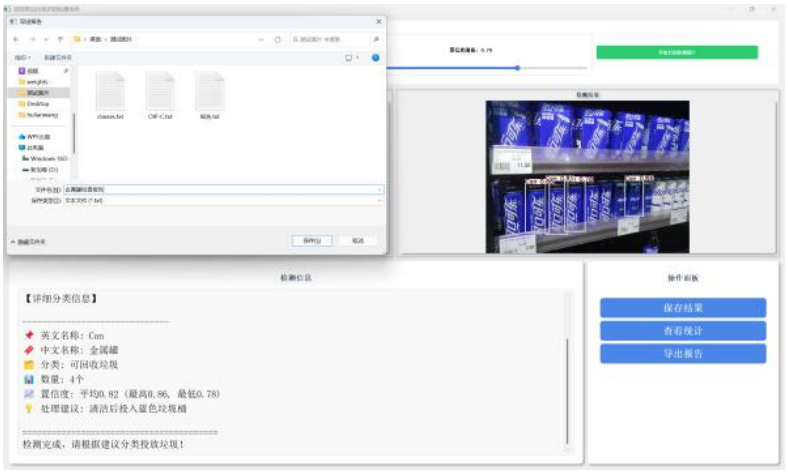


图 16 结果保存

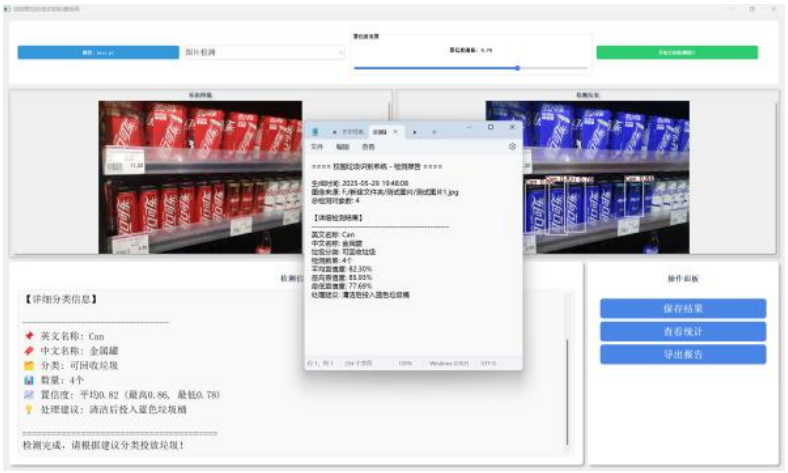


图 17 结果显示

功能说明：根据数据集类别分布（如 Plastic Bottle 占 11.26%）计算反比例权重，缓解类别不平衡问题；权重通过 `Focal Loss` 和 `mlflow` 日志记录，提升小样本类别的训练效果。

4.5 训练优化模块

使用自定义损失函数引入类别权重平衡数据分布；应用多种 Albumentations 变换增强数据策略；使用 MLflow 进行实验跟踪；最后模型评估记录 mAP 等关键指标。

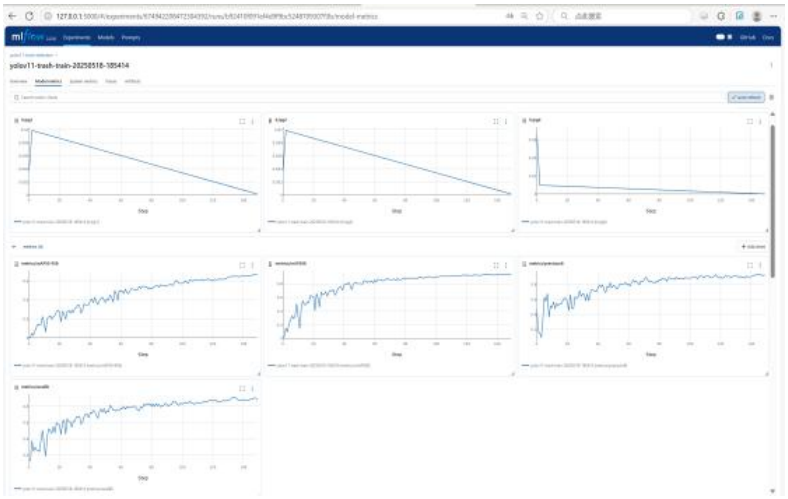


图 18 mlflow监控模型训练过程1

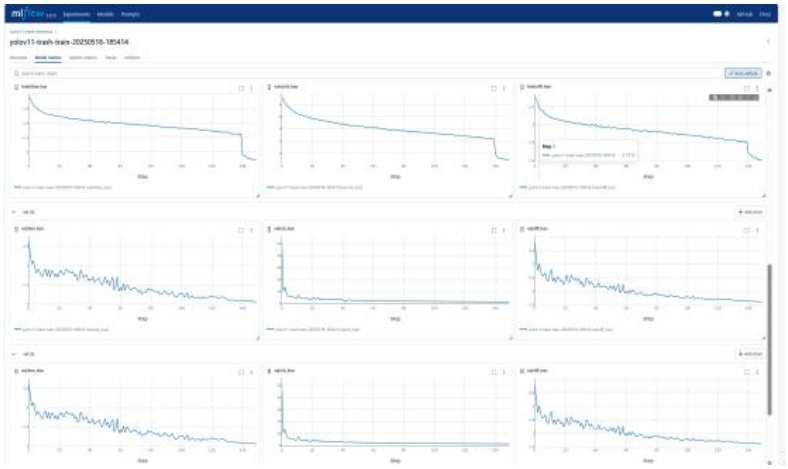


图 19 mlflow监控模型训练过程2

功能说明：实现 **Focal Loss** 改进分类损失，缓解类别不均衡问题；通过 `Adam` 优化器、学习率衰减（`lr0` → `lrf`）、L2 正则（`weight_decay`）提升训练稳定性；对边界框回归（`box`）、分类（`cls`）、DFL（`dfl`）损失分配不同权重。

4.6 系统管理模块

（1） MainWindow 类及其相关方法

解决问题:

该部分为应用的主控制器，负责协调各个部分。其包含了启动/停止检测任

务的入口、处理用户交互、管理工作线程的生命周期以及更新 UI 状态的逻辑。

具体内容:

start_detection(),detect_image(), start_webcam(): 处理用户点击“开始检测”或“启动摄像头”按钮的逻辑，创建并启动相应的工作线程。

stop_detection(),stop_webcam(): 处理用户点击“停止摄像头”或应用需要停止检测时的逻辑，安全地终止工作线程。

closeEvent():在应用关闭时被调用，确保所有后台线程和资源被正确释放。

on_detect_type_changed():管理 UI 状态（按钮可见性、操作面板切换）以与当前的检测模式（图片或摄像头）同步。

update_conf_threshold():管理置信度阈值设置的同步，并在摄像头模式下应用新设置（通过重启 worker）。

连接工作线程的信号到 UI 更新槽函数：例如，将 frame_ready 和 detection_complete 信号连接到 display_webcam_result 和 update_detection_info，实现后台处理结果到前台 UI 的同步。

错误处理相关的槽函数 (handle_error) 和通过 QMessageBox 显示消息的逻辑。

（2） WebcamWorker 和 DetectionWorker 类

解决问题:

这两个是 QThread 的子类，它们将耗时的检测任务从主 UI 线程分离出来执行。它们负责具体任务（采集帧或加载图片、进行检测）的执行、管理自身线程的生命周期、资源（如摄像头）的获取和释放，并通过信号报告进度、结果和错误。

具体内容:

__init__():初始化线程，加载模型，设置参数（如置信度）。

run():线程的主执行循环，包含任务的具体逻辑（如摄像头帧采集和处理循环、图片检测）。

stop():提供一个方法来接收停止信号，并设置内部标志使 run 方法能够安全退出。

资源管理：WebcamWorker 中的_init_camera()和 cap.release()负责摄像头的打

开和释放。

信号定义(pyqtSignal)：frame_ready,detection_complete,error_occurred 等，用于与主线程通信，同步状态和结果。

（3） 错误处理机制 (try...except 块和 QMessageBox):

解决问题:

捕获代码执行过程中可能出现的异常（如文件不存在、模型加载失败、摄像头无法打开、计算错误等），防止程序崩溃，并以用户友好的方式（如消息框或信息面板）通知用户。

具体内容:

代码在多个关键操作（模型加载、图片读取、检测循环、文件保存、报告导出）周围使用了 try...except 结构来捕获异常；通过 QMessageBox.critical()或 self.info_text.append()将错误信息呈现给用户。

（4） 状态同步机制 (PyQt 信号与槽, 控件状态管理):

解决问题:

确保后台线程的处理状态能够正确反映到用户界面上，用户操作也能正确触发后台任务。这是通过信号与槽机制、以及显式地启用/禁用或显示/隐藏 UI 控件来实现的。

具体内容:

线程发射信号，主线程的槽函数更新 UI 控件（如 QLabel 显示图像，QTextEdit 显示文本信息）。

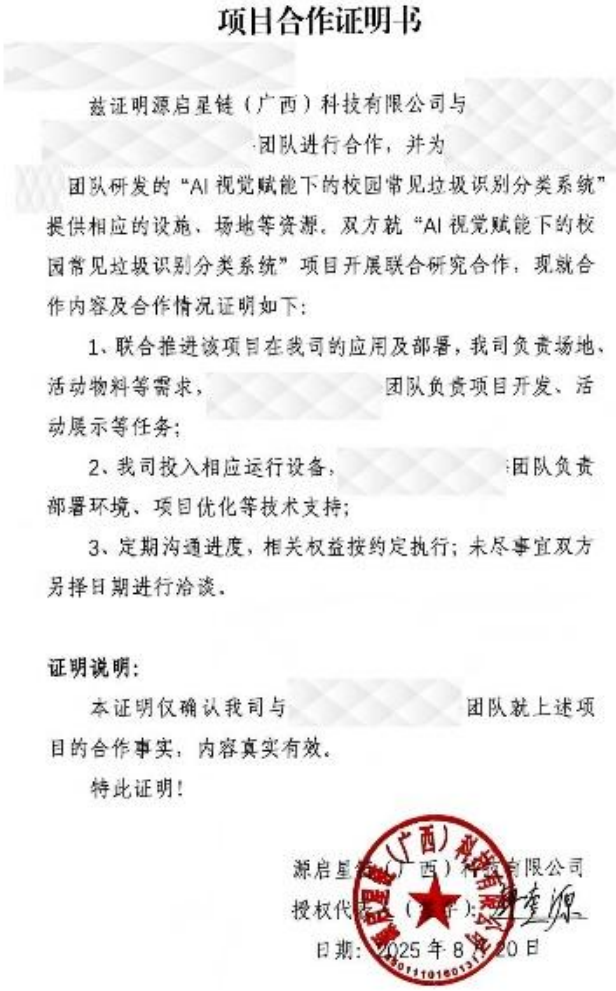
根据检测模式 (detect_type_combo.currentIndexChanged 处理函数)，切换显示不同的操作按钮面板 (image_actions_widget, webcam_actions_widget)。

根据任务状态（是否正在检测），启用或禁用相关的操作按钮（如“开始检测”、“保存结果”等）。

第 5 章 落地案例

本项目自 2025 年睿抗机器人开发者大赛省赛起开始进行研发与优化，并在

获得赛区一等奖后陆续与全国多家单位进行合作，具体落地合作详情如下：



项目合作证明书

兹证明源启星链（广西）科技有限公司与

团队进行合作，并为

团队研发的“AI 视觉赋能下的校园常见垃圾识别分类系统”提供相应的设施、场地等资源。双方就“AI 视觉赋能下的校园常见垃圾识别分类系统”项目开展联合研究合作，现就合作内容及合作情况证明如下：

1、联合推进该项目在我司的应用及部署，我司负责场地、活动物料等需求，团队负责项目开发、活动展示等任务；

2、我司投入相应运行设备，团队负责部署环境、项目优化等技术支持；

3、定期沟通进度，相关权益按约定执行；未尽事宜双方另择日期进行洽谈。

证明说明：

本证明仅确认我司与团队就上述项目的合作事实，内容真实有效。

特此证明！

源启星[illegible]（广西）科技有限公司

授权代[illegible]（签字）：

日期：2025 年 8 月 20 日

5.1 源启星链（广西）科技有限公司

5.2 上海势曦苑科技有限公司

项目合作证明书

团队：

兹证明上海势曦苑科技有限公司与 进行合作，并为桂电智眼分类引擎团队研发的“AI 视觉赋能下的校园常见垃圾识别分类系统”提供相应的设施、场地等资源。双方就“AI 视觉赋能下的校园常见垃圾识别分类系统”项目开展联合研究合作，现就合作内容及合作情况证明如下：

1、联合推进该项目在我司的应用及部署，我司负责场地、活动物料等需求 团队负责项目开发、活动展示等任务；

2、我司投入相应运行设备， 团队负责部署环境、项目优化等技术支持；

3、定期沟通进度，相关权益按约定执行；未尽事宜双方另择日期进行洽谈。

证明说明：

本证明仅确认我司与 团队就上述项目的合作事实，内容真实有效。

特此证明！

上海势曦苑科技有限公司

2025 年 9 月 13 日

5.3 温州富兴阀门有限公司

项目合作证明书

团队：

兹证明 鹿寨乾元德信科技服务中心与

团队进行合作，并为该团队研发“AI 视觉赋能下的校园常见垃圾识别分类系统”提供相应的设施、场地等资源。双方就“AI 视觉赋能下的校园常见垃圾识别分类系统”项目开展联合研究合作，现就合作内容及合作情况证明如下：

1、联合推进该项目在本单位的应用及部署，我单位负责场地、活动物料等需求，贵团队负责项目开发、活动展示等任务；

2、我部门投入相应运行设备，贵团队负责部署环境、项目优化等技术支持；

3、定期沟通进度，相关权益按约定执行；未尽事宜双方另择日期进行洽谈。

证明说明：

本证明仅确认我部门与贵团队就上述项目的合作事实，内容真实有效。

特此证明！

鹿寨乾元德信科技服务中心

授权代表人（签章）

日期：2025 年 12 月 15 日

5.4 鹿寨乾元德信科技服务中心

项目合作证明书

[redacted]团队：

兹证明 鹿寨乾元德信科技服务中心与[redacted]团队进行合作，并为该团队研发“AI 视觉赋能下的校园常见垃圾识别分类系统”提供相应的设施、场地等资源。双方就“AI 视觉赋能下的校园常见垃圾识别分类系统”项目开展联合研究合作，现就合作内容及合作情况证明如下：

1、联合推进该项目在本单位的应用及部署，我单位负责场地、活动物料等需求，贵团队负责项目开发、活动展示等任务；

2、我部门投入相应运行设备，贵团队负责部署环境、项目优化等技术支持；

3、定期沟通进度，相关权益按约定执行；未尽事宜双方另择日期进行洽谈。

证明说明：

本证明仅确认我部门与贵团队就上述项目的合作事实，内容真实有效。

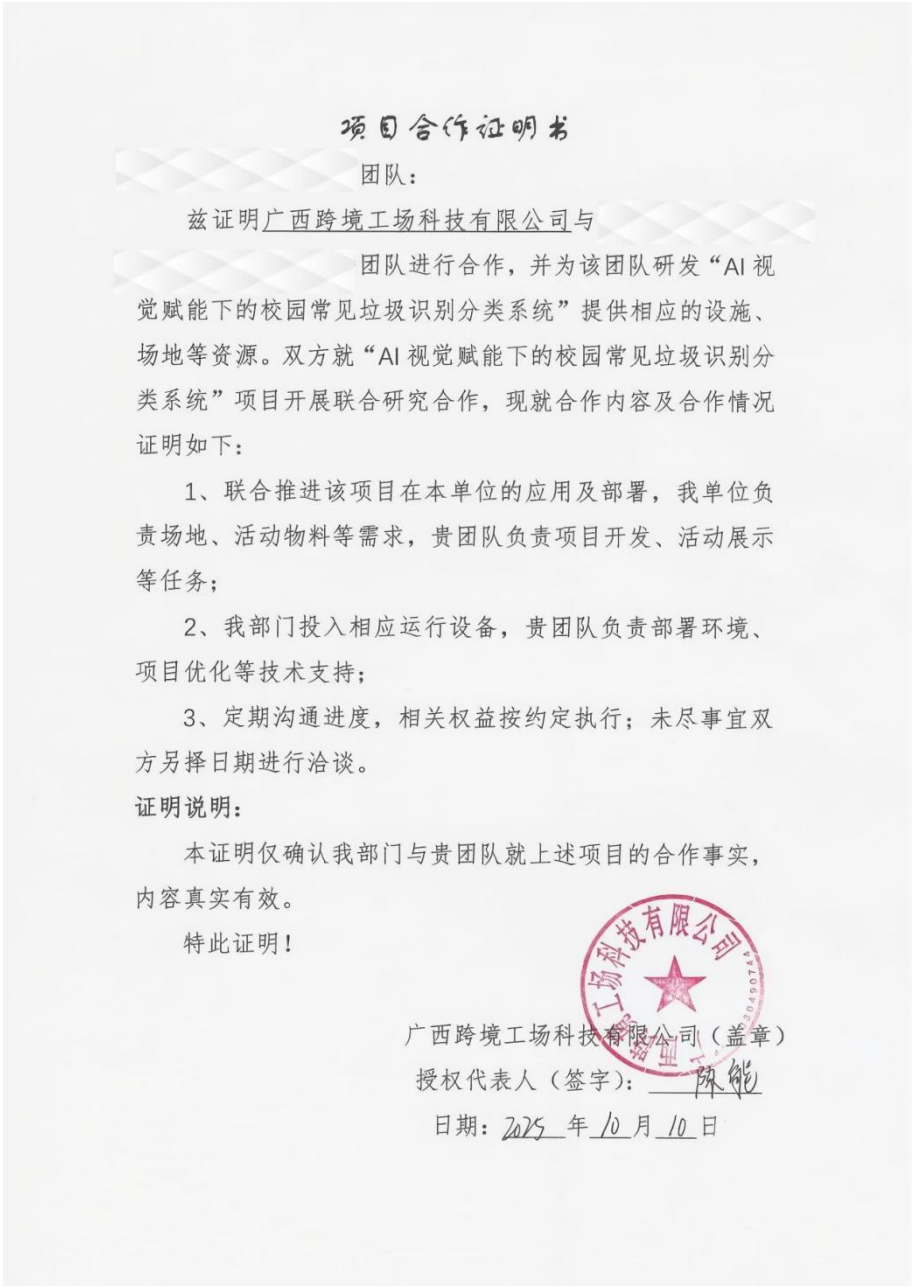
特此证明！

鹿寨乾元德信科技服务中心

授权代表人（签章）

日期：2025 年 12 月 15 日

5.5 广西跨境工厂科技有限公司



项目合作证明书

团队：

兹证明广西跨境工场科技有限公司与

团队进行合作，并为该团队研发“AI视觉赋能下的校园常见垃圾识别分类系统”提供相应的设施、场地等资源。双方就“AI视觉赋能下的校园常见垃圾识别分类系统”项目开展联合研究合作，现就合作内容及合作情况证明如下：

1、联合推进该项目在本单位的应用及部署，我单位负责场地、活动物料等需求，贵团队负责项目开发、活动展示等任务；

2、我部门投入相应运行设备，贵团队负责部署环境、项目优化等技术支持；

3、定期沟通进度，相关权益按约定执行；未尽事宜双方另择日期进行洽谈。

证明说明：

本证明仅确认我部门与贵团队就上述项目的合作事实，内容真实有效。

特此证明！

广西跨境工场科技有限公司（盖章）

授权代表人（签字）：陈能

日期：2025 年 10 月 10 日

5.6 北海市环境卫生管理处

合作框架协议

甲方：[redacted]学院

乙方：北海市环境卫生管理处

经甲乙双方友好协商，本着“优势互补，共同发展，合作双赢”的原则，在平等自愿的前提下，双方同意在实验实践基地方面建立合作关系，并达成如下协议：

一、合作原则

(一)共同参与、科学发展原则:遵循科研规律，实现共同发展。

(二)平等互利原则:甲、乙双方友好合作，充分发挥各自优势，加强合作，共同发展。

二、合作内容

(一)实验实践基地合作

整合甲、乙双方的优势资源，结合单位研究需要，共同建立关于生活垃圾分类相关技术领域用于学生实验研究的实践基地，即甲方自主开发的“校园生活垃圾分类识别科普系统”应用于乙方的“北海市生活垃圾分类宣教基地”中。其中该“校园生活垃圾分类识别科普系统”主要有三个功能模块设置：一是识别检测模块：即现场实时垃圾识别检测，可以让参观者进行实时的生活垃圾分类识别操作；二是互动答题模块：即现场可进行生活垃圾分类科普知识互动问答；三是问答小助手：即关于垃圾分类 AI 问答助手，进行实时人机互动，进行实时的生活垃圾分类科普知识宣传。乙方为甲方开发“校园生活垃圾分类识别科普系统”的便利免费开放北海市生活垃圾分类宣教基

1

地作为实验实践基地使用，不对甲方开发研究进行经费投入。

(二)其它合作事项，根据双方实际需要协商开展。

三、资源共享，全面合作。合作期间，实现资源共享、互利互惠、全面合作。

甲方为乙方无偿开放有关学生实验的过程及成果等资源，甲方开发“校园生活垃圾分类识别科普系统”的过程中可以使用北海市生活垃圾分类宣教基地的相关设施设备，但负责使用相关设施设备的完好性及安全责任。

四、建立定期访问机制。双方建立互访机制，加强合作，增进友谊，协调解决合作中的有关重大问题和具体事宜。

五、由双方共同合作研发的科研成果、工艺及产品等成果共享，未经双方协商同意，任何一方不得私自泄漏或转让第三方。

六、双方商定的科技协作项目等将另行签订专项协议或合同，明确双方的责任、权利和义务，确保各项合作项目能顺利开展。

七、本协议一式两份，自甲、乙双方签字盖章之日起生效，有效期壹年，到期后根据合作情况，经友好协商后再进行续签。

八、本协议未尽事宜，双方协商解决。

甲方:(

乙方:(盖章)

甲方代表签字:

乙方代表签字:

2025年10月16日

2025年10月16日

第 6 章 作品总结

本作品是一款“AI 赋能下的轻量化常见垃圾识别分类系统”，聚焦垃圾分类领域的现实痛点与技术需求，以轻量化、高适配、强科普为核心定位，形成了从技术研发到场景落地的完整闭环。项目背景源于国家垃圾分类政策推进与传统人工分拣效率低、智能设备成本高的矛盾——传统智能分类设备单台成本超 5 万元，且依赖高性能硬件，而高校等场景中智能垃圾桶覆盖率不足 30%，人工督导成本占运营成本 60%。为此，团队以 YOLOv11 系列模型为核心，结合多技术栈与优化策略，打造出适配边缘设备、兼顾精度与效率的解决方案。

在技术实现层面，作品构建了“数据采集-模型优化-推理部署-结果反馈”的全流程体系。数据采集模块支持静态图像加载与实时视频流采集，通过 OpenCV 驱动摄像头以 60FPS 速率获取帧数据，并借助智能帧缓冲机制提升稳定性；模型推理环节选用 YOLOv11n 轻量化模型，搭配自定义预处理流程与 GPU 加速技术，在 CPU≤2.0GHz、内存≤16GB 的硬件约束下，仍能实现高效检测，对塑料瓶、金属罐等高频垃圾的识别精度达 93%以上；针对数据集类别不平衡问题，团队设计了融合类别权重与 FocalLoss 思想的自定义损失函数，使 URPC 数据集上 mAP50 提升 2.4 个百分点，同时通过 Mosaic 增强、K-means++锚框聚类等策略，缓解垃圾形态杂乱、背景干扰强的问题。此外，系统采用多线程架构分离采集、推理与显示任务，避免界面卡顿，还支持置信度动态调节、检测结果多格式保存（JSON/TXT/PNG）与结构化报告生成，满足不同场景下的使用需求。

在功能与场景适配性上，作品突破了传统识别工具的局限，兼具实用性与教育性。除核心的垃圾识别分类功能外，系统内置垃圾分类知识库，能实时显示检测物中英文名称、所属类别、处理建议等信息，还开发了面向中小学生的互动答题模块与 AI 问答助手，通过 Grad-CAM 数据可视化增强环保认知，推动分类行为从“被动监管”向“主动参与”转变。从落地实践来看，项目已与源启星链（广西）科技有限公司、鹿寨乾元德信科技服务中心、北海市环境卫生管理处等 6 家单位达成合作，在校园、垃圾分类宣教基地等场景部署应用，累计普及人数超 4700 人，不仅降低了人工督导成本，还间接助力生活垃圾资源回收率提升，契合北京市“十四五”规划中生活垃圾无害化处理率 100%的目标。

从创新价值与发展前景来看，作品在经济与社会效益上均有显著突破。技术层面，通过深度可分离卷积、Stem 模块等优化，模型参数量压缩至原 77.9%，

单设备部署成本控制在 500 元内，开源框架与迁移学习的应用减少了开发成本，长期运维中可节省 60%的人工成本，且 2025 年智能垃圾分类市场规模预计超 2000 亿元，项目具备广阔的商业化潜力；社会效益上，系统既能直接服务校园、社区等场景的垃圾分类，还可拓展至交通枢纽等公共区域，未来计划接入自动抓取、回收硬件，构建“识别-分拣-回收”全闭环机器人，进一步推动垃圾分类的智能化与自动化升级，为国家“双碳”战略与绿色低碳教育体系建设提供技术支撑。

第 7 章 作品展望

团队成员在研发期间曾受邀出席 2025 年华为开发者大会进行技术交流，在期间曾参观华为 CloudRobo 智能赋能物流展示，并与深圳市人工智能与机器人研究院常务副院广东省具身智能机器人创新中心主任丁宁探讨这一项目。本团队计划在比赛结束后不断优化项目，并且借助学院硬件设施开始着手接入硬件，如“可自动识别检测-分类-抓取-回收至对应地点”整套自动完成的机器人（狗），或者配置显示器、音响等设备的置于定点垃圾桶的摄像分类检测装置等多种设置。



图 20图 华为CloudRobo智能赋能物流设备

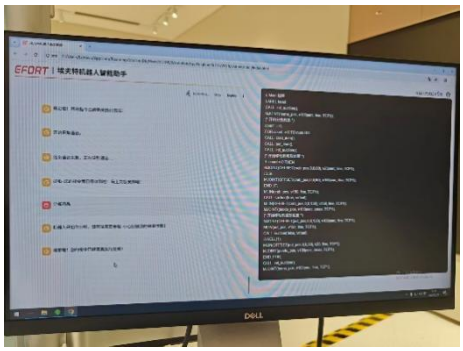


图 22 华为CloudRobo智能赋能物流显示图



图 21 华为ColudRobo智能赋能物流设备2



丁宁 深圳市人工智能与机器人研究院常务副院长
广东省具身智能机器人创新中心主任

图 23 合照

我们希望在我们这一代，科技不仅仅是个体所掌握的技术，也不仅仅是某一公司所用来发展的工具，而是造福万家，为你我，为每一个辛苦劳作的人带去福音，为中国在全世界的发展贡献属于我们自己微不足道的力量。

参考文献

[1]刘玉蕾,褚丽莉,李波.改进 YOLOv11 的模糊监控异常检测算法[J/OL].电子测量技术,1-13[2025-10-22].https://link.cnki.net/urlid/11.2175.tn.20251021.1825.018.

[2]刘伟,钱莉.基于 OpenCV 环境的 SIFT、SURF、ORB 算法比较分析[J].化工自动化及仪表,2018,45(09):714-716+721.DOI:CNKI:SUN:HGZD.0.2018-09-014.

[3]中华人民共和国住房和城乡建设部. 关于加快推进部分重点城市生活垃圾分类工作的通知[EB/OL]. [2017-12-20/2021-01-15].

[4]中华人民共和国教育部.教育部关于印发《高等学校人工智能创新行动计划》的通知 [Z]. 教技〔2018〕3 号，2018-04-03.

[5]中华人民共和国工业和信息化部，中华人民共和国中央网络安全和信息化委员会办公室，中华人民共和国教育部，中华人民共和国国家卫生健康委员会，中国人民银行，国务院国有资产监督管理委员会.工业和信息化部等六部门关于印发《算力基础设施高质量发展行动计划》的通知 [Z]. 工信部联通信〔2023〕180 号，2023-10-08..